

《计算机系统基础》

第7章 链接

授课老师:

何璐璐

luluhe@whu.edu.cn

多重定义符号的解析示例1

以下程序会发生链接出错吗？

```
int x=10;
int p1(void);
int main()
{
    x=p1();
    return x;
}
```

main.c

```
int x=20;
int p1()
{
    return x;
}
```

p1.c

多重定义符号的解析示例1

以下程序会发生链接出错吗？

```
int x=10;
int p1(void);
int main()
{
    x=p1();
    return x;
}
```

main.c

```
int x=20;
int p1()
{
    return x;
}
```

p1.c

main只有一次强定义

p1有一次强定义，一次弱定义

x有两次强定义，所以，链接器
将输出一条出错信息

多重定义符号的解析示例2

以下程序会发生链接出错吗？

```
# include <stdio.h>
int  y=100;
int  z;
void p1(void);
int main()
{
    z=1000;
    p1( );
    printf("y=%d, z=%d\n", y, z);
    return 0;
}
```

main.c

```
int  y;
int  z;
void p1( )
{
    y=200;
    z=2000;
}
```

p1.c

问题：打印结果是什么？

多重定义符号的解析示例2

以下程序会发生链接出错吗？

```
# include <stdio.h>
int  y=100;
int  z;
void p1(void);
int  main()
{
    z=1000;
    p1( );
    printf("y=%d, z=%d\n", y, z);
    return 0;
}
```

main.c

问题：打印结果是什么？

y=200, z=2000

main.o中符号y为唯一定义符号，

p1中的y为引用符号

y一次强定义，一次弱定义；
z两次弱定义；
p1一次强定义，一次弱定义；
main一次强定义。

```
int  y;
int  z;
void p1( )
{
    y=200;
    z=2000;
}
```

p1.c

该例说明：在两个不同模块定义相同变量名，很可能发生意想不到的结果！

多重定义符号的解析举例3

以下程序会发生链接出错吗? `main.c`

```
1  #include <stdio.h>
2  int d=100;
3  int x=200;
4  void p1(void);
5  int main()
6  {
7      p1();
8      printf("d=%d,x=%d\n",d,x);
9      return 0;
10 }
```

`p1.c`

```
1  double d;
2
3  void p1()
4  {
5      d=1.0;
6  }
```

p1执行后d和x处内容是什么?

问题: 打印结果是什么?

多重定义符号的解析举例3

以下程序会发生链接出错吗? `main.c`

```
1  #include <stdio.h>
2  int  d=100;
3  int  x=200;
4  void p1(void);
5  int  main()
6  {
7      p1();
8      printf("d=%x,x=%x\n",d,x);
9      return 0;
10 }
```

问题：打印结果是什么？

d=0, x=0x3FF0 0000

该例说明：两个重复定义的变量具有不同
类型时，更容易出现难以理解的结果！

`p1.c`

```
1  double d;
2
3  void p1()
4  {
5      d=1.0;
6  }
```

&d

p1执行后d和x处内容是什么？

	0	1	2	3
&x	00	00	F0	3F
&d	00	00	00	00

1.0: 0 01111111111 0...0B
=3FF0 0000 0000 0000H
(见双精度格式)

全局变量

- 如果可以，请尽量避免
- 否则
 - 如果可以，请使用 **static**
 - 如果你定义了一个全局变量，请初始化
 - 如果你引用了外部的全局变量，请使用 **extern**

练习

假设一个C 程序有两个源文件：main.c 和test. c ，内容如下：

```
1  /* main.c */
2  int sum();
3
4  int a[4]={1, 2, 3, 4};
5  extern int val;
6  int main( )
7  {
8      val=sum();
9      return val;
10 }
```

```
1  /* test.c */
2  extern int a[];
3  int val=0;
4  int sum()
5  {
6      int i;
7      for (i=0; i<4; i++)
8          val += a[i];
9      return val;
10 }
```

对于编译生成的可重定位目标文件test.o ，填写下表中各符号的情况。

符号	是否在test.o的符号表中	符号类型（全局、局部、外部）	符号定义所在的模块及节
a			
val			
sum			
i			

步骤2：重定位

- 基于符号解析，识别出有关联的目标模块
- 将这些模块中的节进行合并，并确定运行时符号的地址
- 在引用符号的位置，重定位引用地址

步骤2：重定位

1. 节的合并和符号地址的确定

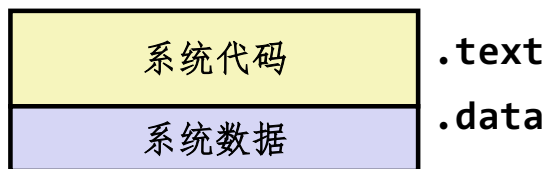
- 将相互关联的所有可重定位文件中相同类型的（如`.text`）节合并成一个同一类型的新节。
 - 例如，所有模块中的`.data`节合并为一个大的`.data`节
- 链接器为新节中的每个定义符号确定存储地址。
- 完成这一步后，每条指令和每个全局变量都可确定地址。

节与定义符号的重定位： 示例1

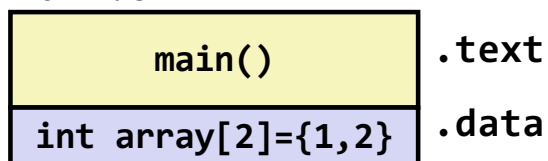
```
int sum(int *a, int n);  
int array[2] = {1, 2};  
int main()  
{  
    int val = sum(array, 2);  
    return val;  
}  
main.c
```

```
int sum(int *a, int n)  
{  
    int i, s = 0;  
    for (i = 0; i < n; i++) {  
        s += a[i];  
    }  
    return s;  
}  
sum.c
```

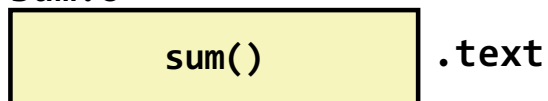
可重定位目标文件



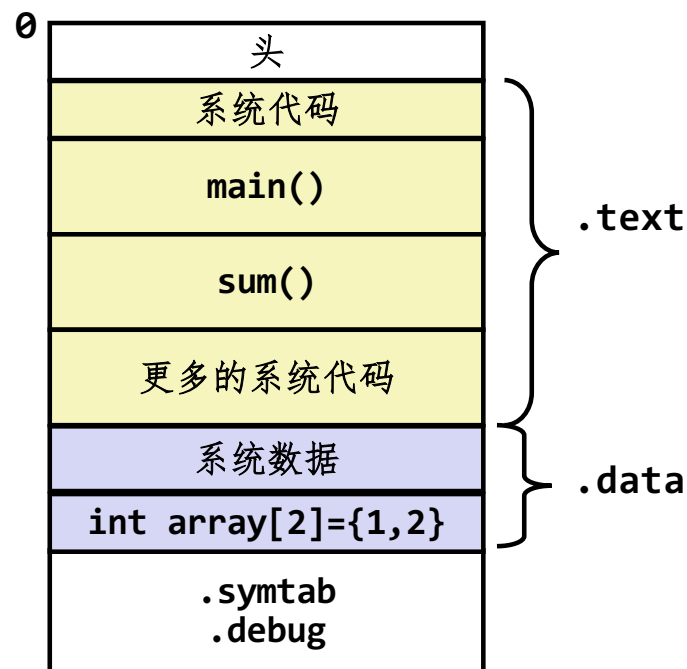
main.o



sum.o



可执行目标文件



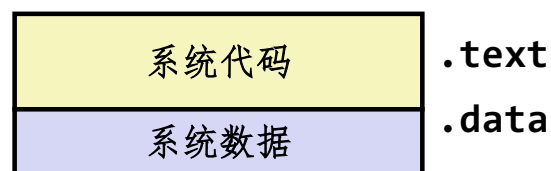
节与定义符号的重定位：示例2

main.c

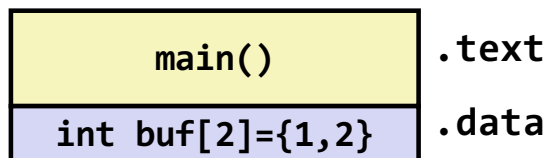
```
int buf[2] = {1, 2};  
void swap();  
  
int main()  
{  
    swap();  
    return 0;  
}
```

```
extern int buf[];  
  
int *bufp0 = &buf[0];  
static int *bufp1;  
  
void swap()  
{  
    int temp;  
  
    bufp1 = &buf[1];  
    temp = *bufp0;  
    *bufp0 = *bufp1;  
    *bufp1 = temp;  
}
```

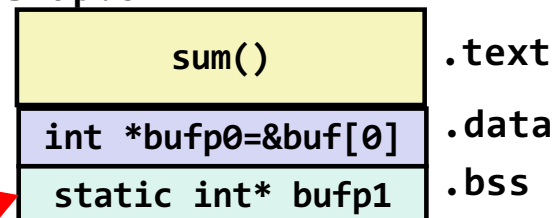
可重定位目标文件



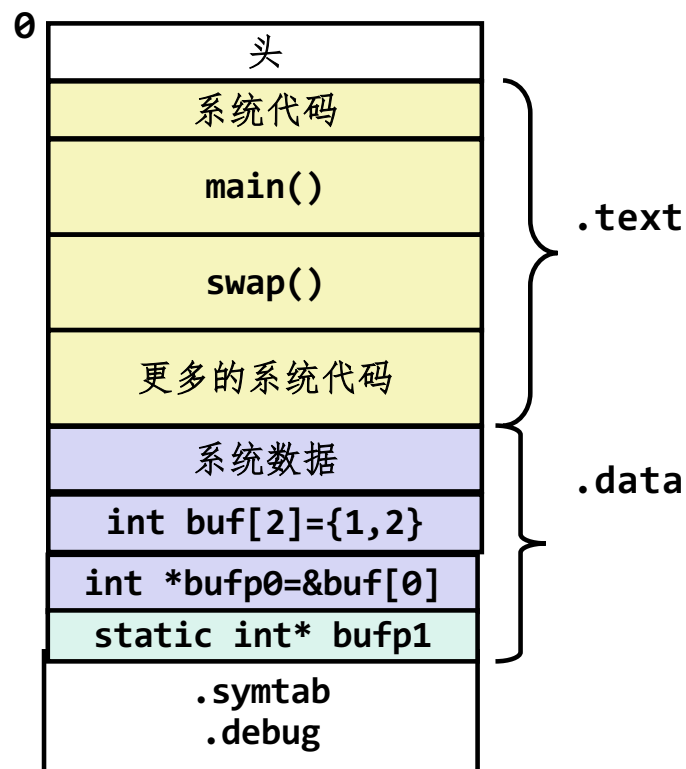
main.o



swap.o



可执行目标文件



虽然是swap的本地符号，也需在.bss节重定位

步骤2：重定位

■ 2. 符号引用的重定位

- 链接器对合并后新代码节和新数据节中的引用符号进行重定位，使其指向对应的定义符号起始处。
- 修改`.text`节和`.data`节中对每个符号的引用（地址）。
- 需要用到在`.rel_data`和`.rel_text`节中保存的**重定位信息**。

可重定位文件 — 链接之前

- 重定位2
 - 借助重定位表，重写寻址字段数据
- 示例：
 - 编译之后链接之前，**main.o**不知道**sum**和**array**的地址
 - 指令中填的是0，需要重定位
 - **main.o**是可重定位文件

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

main.c

```
0000000000000000 <main>:
 0:  48 83 ec 08          sub    $0x8,%rsp
 4:  be 02 00 00 00      mov    $0x2,%esi
 9:  bf 00 00 00 00      mov    $0x0,%edi          # %edi = &array
                          # Relocation entry
                          a: R_X86_64_32 array
 e:  e8 00 00 00 00      callq 13 <main+0x13>      # sum()
                          f: R_X86_64_PC32 sum-0x4
                          # Relocation entry
13:  48 83 c4 08          add    $0x8,%rsp
17:  c3                   retq
                          main.o
```

Source: objdump -r -d main.o

重定位信息

- 汇编器遇到对位置未知的目标引用时，生成一个重定位条目
- 数据（指令）引用的重定位条目在 `.rel_data` (`.rel_text`) 节中
- ELF中重定位条目格式如下：

```
typedef struct {  
    long  offset;           /*需重定位的引用的节偏移*  
    long  type:32,          /*重定位类型（即修改方式）*/  
        symbol:32;         /*需重定位的引用所指向的符号*/  
    long addend;            /*某些重定位类型需要用到的常数偏移量*/  
}Elf64_Rela;
```

- 重定位类型与处理器有关，最基本的两种重定位类型
- **R_386_PC32**：指明引用采用**PC**相对寻址方式，即有效地址为**PC**内容加上重定位后的地址，**PC**为下条指令地址
- **R_386_32**：使用**32**位绝对地址

重定位条目

```
int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}                                     main.c
```

待重定位引用的位置

0000000000000000 <main>:	
0:	48 83 ec 08 sub \$0x8,%rsp
4:	be 02 00 00 00 mov \$0x2,%esi
9:	bf 00 00 00 00 mov \$0x0,%edi # %edi = &array
	a: R_X86_64_32 array # 重定位条目
e:	e8 00 00 00 00 callq 13 <main+0x13> # sum()
	f: R_X86_64_PC32 sum-0x4 # 重定位条目
13:	48 83 c4 08 add \$0x8,%rsp
17:	c3 retq
	main.o

32位绝对地址的引用

32位PC相对地址的引用

Source: objdump -r -d main.o

重定位条目

```
typedef struct {
    long offset;           /*需重定位的引用的节偏移*/
    long type:32,          /*重定位类型 (即修改方式)*/
        symbol:32;        /*需重定位的引用所指向的符号*/
    long addend;           /*某些重定位类型需要用到的常数偏移量*/
}Elf64_Rela;
```

addend:

- 表示与下一条指令地址 (PC)的距离
- 主要用于PC相对寻址

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

```
rrerre@Lenovo-PC:~/csapp/chap7$ readelf -r main.o
```

```
Relocation section '.rela.text' at offset 0x1e8 contains 2 entries:
  Offset             Info           Type           Sym. Value      Sym. Name + Addend
000000000000a 000900000000a R_X86_64_32      0000000000000000 array + 0
000000000000f 000a000000002 R_X86_64_PC32    0000000000000000 sum - 4
```

main.o

Source: objdump -r -d main.o

PC相对寻址

```
0:  48 89 f8          mov    %rdi,%rax
3:  eb 03             jmp    8<loop+0x8>
5:  48 d1 f8          sar    %rax
8:  48 85 c0          test   %rax,%rax
b:  7f f8             jg     5<loop+0x5>
d:  f3 c3             repz  retq
```

- Line 2: eb 03 → jmp 8?
- Line 5: 7f f8 → jg 5?
- PC relative encodings for jump-targets (???):
 - encodings + PC = jump-target
 - 03 + 5 = 8
 - f8(-8) + d = 5

PC相对寻址示例——对sum函数的引用

```
00000000004004d0 <main>:
 4004d0: 48 83 ec 08      sub    $0x8,%rsp
 4004d4: be 02 00 00 00    mov    $0x2,%esi
 4004d9: bf 18 10 60 00    mov    $0x601018,%edi    # %edi = &array
 4004de: e8 05 00 00 00    callq 4004e8 <sum>      # sum()
4004e3: 48 83 c4 08      add    $0x8,%rsp
 4004e7: c3              retq

00000000004004e8 <sum>:
4004e8: b8 00 00 00 00    mov    $0x0,%eax
 4004ed: ba 00 00 00 00    mov    $0x0,%edx
 4004f2: eb 09            jmp    4004fd <sum+0x15>
```

```
rrerre@Lenovo-PC: ~/csapp/chap7$ readelf -r main.o
```

Relocation section '.rela.text' at offset 0x1e8 contains 2 entries:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
000000000000a	000900000000a	R_X86_64_32	0000000000000000	array + 0
000000000000f	000a000000002	R_X86_64_PC32	0000000000000000	sum - 4

待修改的引用位置 $\text{refAddr} = \text{main.address} + \text{rela.offset} = 0x4004d0 + 0xf = 0x4004df$
执行 `callq sum` 指令时, 当前PC为: $\text{refAddr} - \text{rela.addend} = 0x4004df - (-4) = 0x4004e3$
待修改的新值为: $\text{sum.addr} - \text{当前PC} = 0x4004e8 - 0x4004e3 = 0x5$ (小端模式)

Source: `objdump -dx prog`

重定位条目（绝对寻址）——链接之前

relocation entry for array at 0xa:

offset = 0xa;

addend = 0;

symbol = array;

type = Abs32

00000000004004d0 <main>:

array的绝对地址: 0x601018

```
int sum(int *a, int n);

int array[2] = {1, 2};

int main()
{
    int val = sum(array, 2);
    return val;
}
```

0000000000000000 <main>:

```
0:  48 83 ec 08          sub    $0x8,%rsp
4:  be 02 00 00 00        mov    $0x2,%esi
9:  bf 00 00 00 00        mov    $0x0,%edi    # %edi = &array
                        # Relocation entry
                        a: R_X86_64_32 array
```

rrerre@Lenovo-PC:~/csapp/chap7\$ readelf -r main.o

Relocation section '.rel.text' at offset 0x1e8 contains 2 entries:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
000000000000000a	000900000000a	R_X86_64_32	0000000000000000	array + 0
000000000000000f	000a000000002	R_X86_64_PC32	0000000000000000	sum - 4

Source: objdump -r -d main.o

PC相对寻址示例——对array的引用

array的绝对地址: 0x601018 (小端)

```
00000000004004d0 <main>:
 4004d0: 48 83 ec 08      sub    $0x8,%rsp
 4004d4: be 02 00 00 00    mov    $0x2,%esi
 4004d9: bf 18 10 60 00    mov    $0x601018,%edi # %edi = &array
 4004de: e8 05 00 00 00    callq 4004e8 <sum>    # sum()
 4004e3: 48 83 c4 08      add    $0x8,%rsp
 4004e7: c3              retq

00000000004004e8 <sum>:
 4004e8: b8 00 00 00 00    mov    $0x0,%eax
 4004ed: ba 00 00 00 00    mov    $0x0,%edx
 4004f2: eb 09            jmp     4004fd <sum+0x15>
 4004f4: 48 63 ca        movslq %edx,%rcx
 4004f7: 03 04 8f        add    (%rdi,%rcx,4),%eax
 4004fa: 83 c2 01        add    $0x1,%edx
 4004fd: 39 f2           cmp    %esi,%edx
 4004ff: 7c f3           jl     4004f4 <sum+0xc>
 400501: f3 c3          repz  retq
```

Source: objdump -dx prog

```

1  00000000004004d0 <main>:
2  4004d0:  48 83 ec 08          sub    $0x8,%rsp
3  4004d4:  be 02 00 00 00      mov    $0x2,%esi
4  4004d9:  bf 18 10 60 00      mov    $0x601018,%edi    %edi = &array
5  4004de:  e8 05 00 00 00      callq 4004e8 <sum>      sum()
6  4004e3:  48 83 c4 08          add    $0x8,%rsp
7  4004e7:  c3                  retq

8  00000000004004e8 <sum>:
9  4004e8:  b8 00 00 00 00      mov    $0x0,%eax
10 4004ed:  ba 00 00 00 00      mov    $0x0,%edx
11 4004f2:  eb 09              jmp    4004fd <sum+0x15>
12 4004f4:  48 63 ca          movslq %edx,%rcx
13 4004f7:  03 04 8f          add    (%rdi,%rcx,4),%eax
14 4004fa:  83 c2 01          add    $0x1,%edx
15 4004fd:  39 f2             cmp    %esi,%edx
16 4004ff:  7c f3             jl     4004f4 <sum+0xc>
17 400501:  f3 c3            repz retq

```

a) 已重定位的.text 节

```

1  0000000000601018 <array>:
2  601018:  01 00 00 00 02 00 00 00

```

b) 已重定位的.data 节

图 7-12 可执行文件 prog 的已重定位的.text 节和.data 节。原始的 C 代码在图 7-1 中

练习题 7.4 本题是关于图 7-12a 中的已重定位程序的。

- A. 第 5 行中对 sum 的重定位引用的十六进制地址是多少？
- B. 第 5 行中对 sum 的重定位引用的十六进制值是多少？

A: 0x4004df

B: 0x05

code/link/m.c	code/link/swap.c
<pre> 1 void swap(); 2 3 int buf[2] = {1, 2}; 4 5 int main() 6 { 7 swap(); 8 return 0; 9 }</pre>	<pre> 1 extern int buf[]; 2 3 int *bufp0 = &buf[0]; 4 int *bufp1; 5 6 void swap() 7 { 8 int temp; 9 10 bufp1 = &buf[1]; 11 temp = *bufp0; 12 *bufp0 = *bufp1; 13 *bufp1 = temp; 14 }</pre>
code/link/m.c	code/link/swap.c

a) m.c

b) swap.c

练习题 7.5 考虑目标文件 m.o 中对 swap 函数的调用(图 7-5)。

```
9:  e8 00 00 00 00      callq  e <main+0xe>      swap()
```

它的重定位条目如下:

```

r.offset = 0xa
r.symbol = swap
r.type   = R_X86_64_PC32
r.addend = -4
```

现在假设链接器将 m.o 中的 .text 重定位到地址 0x4004d0, 将 swap 重定位到地址 0x4004e8。那么 callq 指令中对 swap 的重定位引用的值是什么?

0xa

PIC: 位置无关代码

```
ubuntu@ip-172-31-22-210:~/test$ readelf -r symbol.o
```

Relocation section '.rela.text' at offset 0x268 contains 2 entries:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
000000000014	000900000002	R_X86_64_PC32	0000000000000000	array - 4
00000000001e	000c00000004	R_X86_64_PLT32	0000000000000000	sum - 4

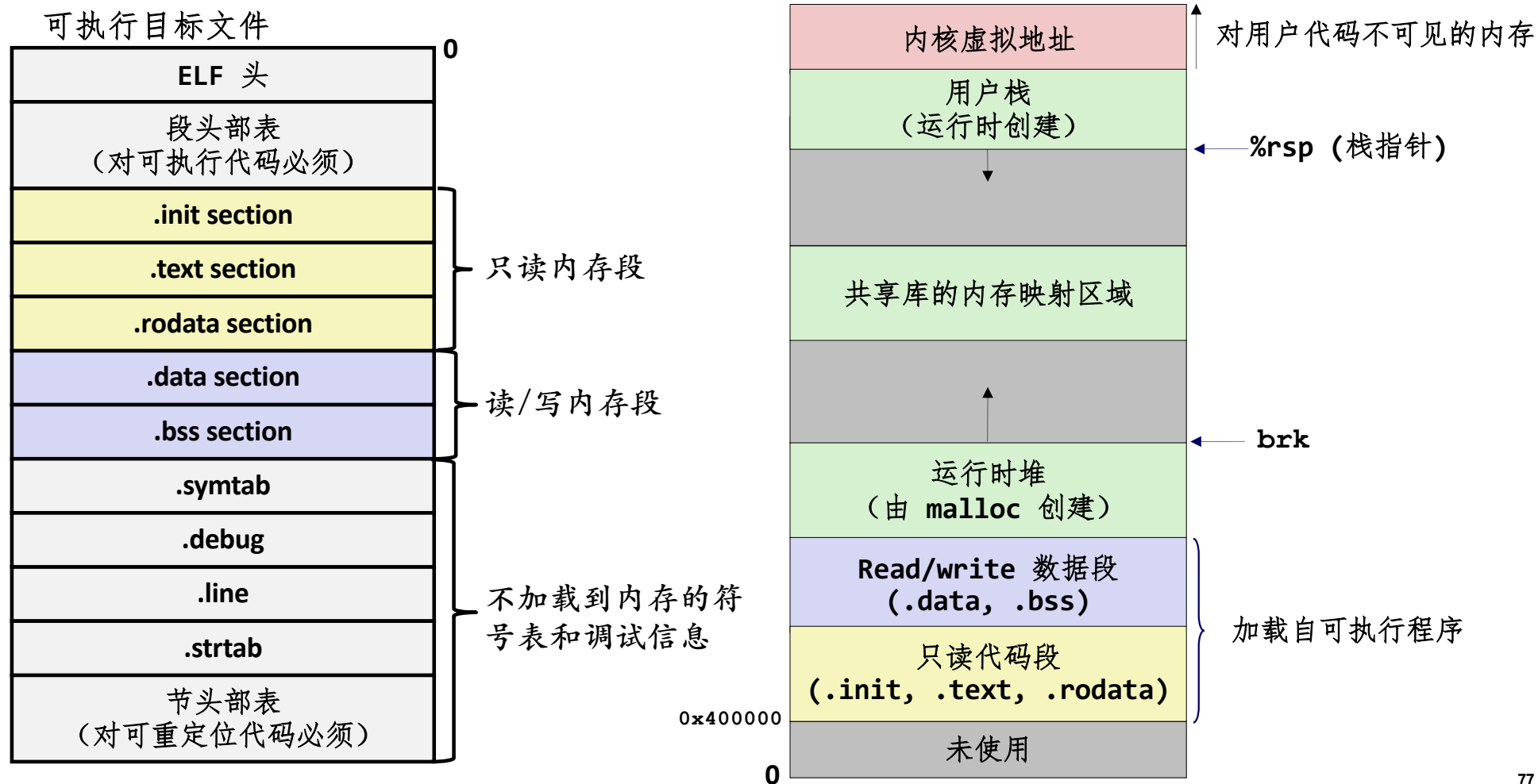
Relocation section '.rela.eh_frame' at offset 0x298 contains 1 entry:

Offset	Info	Type	Sym. Value	Sym. Name + Addend
000000000020	000200000002	R_X86_64_PC32	0000000000000000	.text + 0

```
linux> objdump -dx main.o
```

```
0000000000000000 <main>:
 0: 55                push    %rbp
 1: 48 89 e5          mov     %rsp,%rbp
 4: 48 83 ec 10       sub     $0x10,%rsp
 8: be 02 00 00 00    mov     $0x2,%esi
d: 48 8d 3d 00 00 00 00 lea     0x0(%rip),%rdi    # 14 <main+0x14>
                        10: R_X86_64_PC32    array-0x4
14: e8 00 00 00 00    callq   19 <main+0x19>
                        15: R_X86_64_PLT32    sum-0x4
19: 89 45 fc          mov     %eax,-0x4(%rbp)
1c: 8b 45 fc          mov     -0x4(%rbp),%eax
1f: c9               leaveq
20: c3               retq
```

加载可执行目标文件



打包常用函数

- 如何将程序员常用的函数打包到一起？
 - 数学，I/O，内存管理，字符串处理，等等
- 到目前为止，有两种方式：
 - **方式1：将所有的函数都放进一个单一的源文件中**
 - 程序员把这个大的目标文件链接进他们的程序中
 - 空间和时间效率都很低
 - **linux> gcc main.c /usr/lib/libc.o**
 - **方式2：将每个函数放在一个独立的原文件中**
 - 程序员显示地将适当的二进制文件链接进他们的程序中
 - 更有效，但对程序员来说增加了负担
 - **linux> gcc main.c /usr/lib/printf.o /usr/lib/scanf.o**

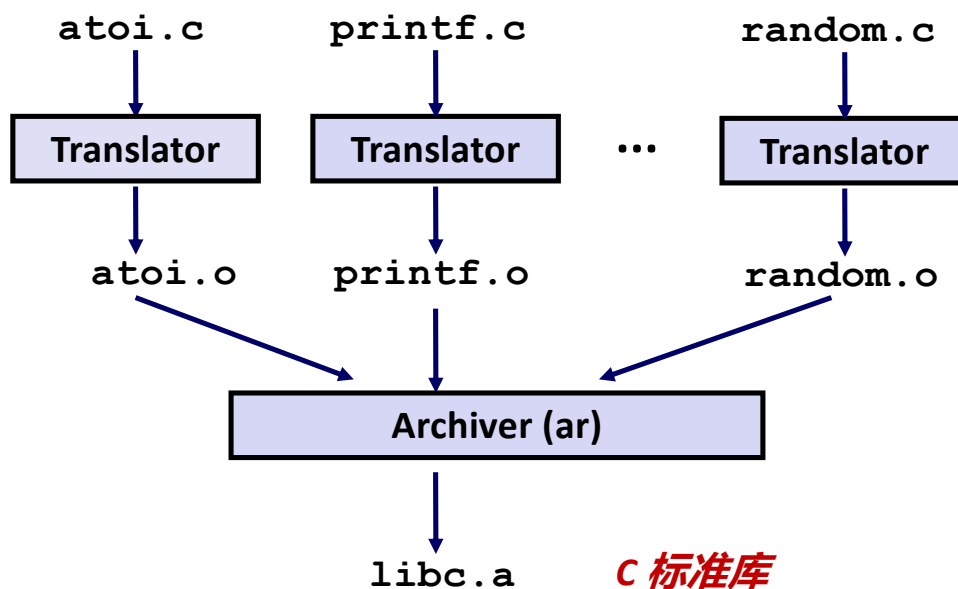
以前的解决方案：静态库

■ 静态库（.a 存档文件）

- 将一些相关联的可重定位目标文件连接成一个带索引的文件（称为**存档文件** *archive*）
- 改进链接器，使它可以在一个或多个存档文件中寻找符号，以解析未被解析的外部引用
- 如果某个存档文件能够解析某个引用，就将它链接到可执行文件中（把用到的模块从库中拷贝出来）
- 例如：使用C标准库和数学库中函数的程序可以用形式如下的命令行来编译和链接：
 - **`linux> gcc main.c /usr/lib/libm.a /usr/lib/libc.a`**
- 在gcc命令中无需明显指定C标准库**libc.a**(默认库)

创建静态库

```
linux> ar rcs libc.a atoi.o printf.o ... random.o
```



- **Archiver** 允许增量更新
- 重新编译发生了变化的函数，在存档文件中替换相应的 `.o` 文件

常见的库

libc.a (C 标准库)

- 4.6 MB 存档了 1496 个目标文件
- I/O, 内存分配, 信号处理, 字符串处理, 数据和时间, 随机数, 整数数学

libm.a (C 数学库)

- 2 MB 存档了 444 个目标文件
- 浮点数数学 (sin, cos, tan, log, exp, sqrt, ...)

```
% ar -t libc.a | sort
...
fork.o
...
fprintf.o
fpu_control.o
fputc.o
freopen.o
fscanf.o
fseek.o
fstab.o
...
```

```
% ar -t libm.a | sort
...
e_acos.o
e_acosf.o
e_acosh.o
e_acoshf.o
e_acoshl.o
e_acosl.o
e_asin.o
e_asinf.o
e_asinl.o
...
```

与静态库链接

addvec.c

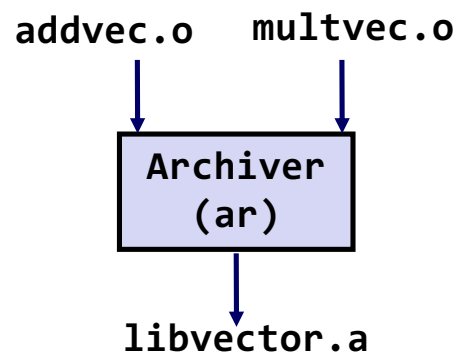
```
void addvec(int *x, int *y,  
            int *z, int n) {  
    int i;  
  
    for (i = 0; i < n; i++)  
        z[i] = x[i] + y[i];  
}
```

multvec.c

```
void multvec(int *x, int *y,  
             int *z, int n)  
{  
    int i;  
  
    for (i = 0; i < n; i++)  
        z[i] = x[i] * y[i];  
}
```

linux> gcc -c addvec.c multvec.c

linux> ar rcs libvector.a addvec.o multvec.o



与静态库链接

libvector.a



addvec.c

```
void addvec(int *x, int *y,
            int *z, int n) {
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] + y[i];
}
```

multivec.c

```
void multivec(int *x, int *y,
              int *z, int n)
{
    int i;

    for (i = 0; i < n; i++)
        z[i] = x[i] * y[i];
}
```

vector.h

```
void addvec(int *x, int *y, int *z, int n);
void multivec(int *x, int *y, int *z, int n);
```

```
#include <stdio.h>
#include "vector.h"
```

```
int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];
```

```
int main()
{
    addvec(x, y, z, 2);
    printf("z = [%d %d]\n", z[0], z[1]);
    return 0;
}
```

main2.c

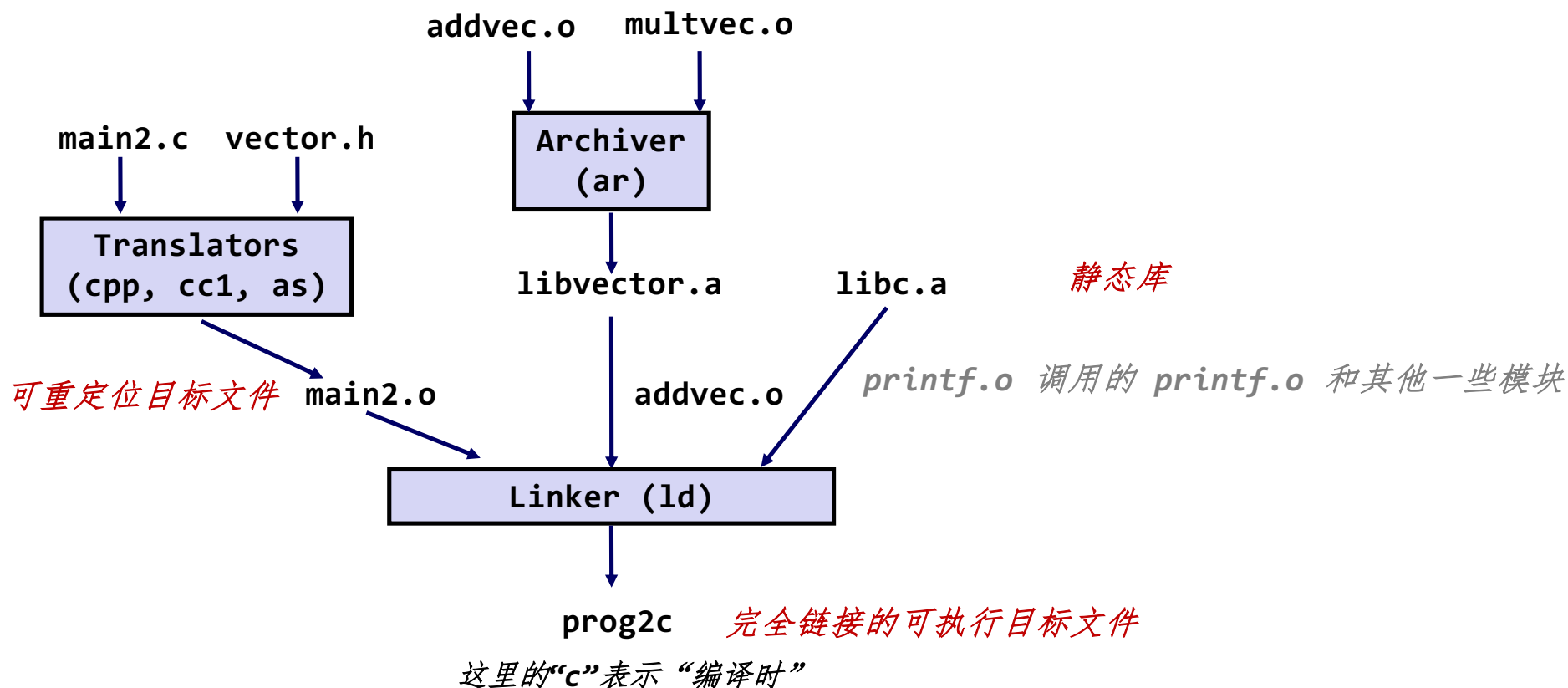
```
linux> gcc -c main2.c
```

```
linux> gcc -static -o prog2c main2.o ./libvector.a
```


与静态库链接

```
linux> gcc -c main2.c
```

```
linux> gcc -static -o prog2c main2.o ./libvector.a
```



与静态库链接

■ 链接器用来解析外部引用的算法

- 按照命令行顺序扫描 **.o** 文件和 **.a** 文件
- 在扫描过程中，维护一个当前未解析的引用列表
- 每遇到一个新的 **.o** 或 **.a** 文件，**obj**，就试着用 **obj** 里定义的符号去解析该列表中未解析的引用
- 如果到扫描结束时，这个未解析列表中还有条目，那么就报错

■ 问题

- 命令行顺序很重要！
- 一般规律：将库放在命令行的最后

```
linux> gcc static ./libvector.a main2.c  
/tmp/cc9XH6Rp.o: In function 'main':  
/tmp/cc9XH6Rp.o(.text+0x18): undefined reference to 'addvec'
```

静态库链接顺序准则

- 通常将静态库文件放在命令行文件列表的后面；
- 若有多个静态库文件，则根据这些文件目标模块中的符号引用关系来确定顺序。
 - 若有引用关系，则按照引用顺序放置， 如A→B， `gcc -o func obj.o A B`
 - 若没有引用关系，顺序任意。

静态库链接顺序准则

- 假设调用关系如下：

func.o → **libx.a** 和 **liby.a** 中的函数

libx.a → **libz.a** 中的函数

libx.a 和 **liby.a** 之间、**liby.a** 和 **libz.a** 相互独立

则以下几个命令行都是可行的：

```
linux> gcc -static -o myfunc func.o libx.a liby.a libz.a
```

```
linux> gcc -static -o myfunc func.o liby.a libx.a libz.a
```

```
linux> gcc -static -o myfunc func.o libx.a libz.a liby.a
```

静态库链接顺序准则

- 假设调用关系如下：

func.o $\rightarrow$ ***libx.a*** 和 ***liby.a*** 中的函数

libx.a $\rightarrow$ ***liby.a*** 同时 ***liby.a*** $\rightarrow$ ***libx.a***

则以下命令行可行：

```
linux> gcc -static -o myfunc func.o libx.a liby.a libx.a
```

现在的做法：共享库

■ 静态库的缺点

- 存储的可执行文件中有大量重复（每个函数都需要 `libc`）
- 运行的可执行文件中有大量重复
- 对系统库的很小的 `bug` 修复，要求每个应用程序都进行重新链接

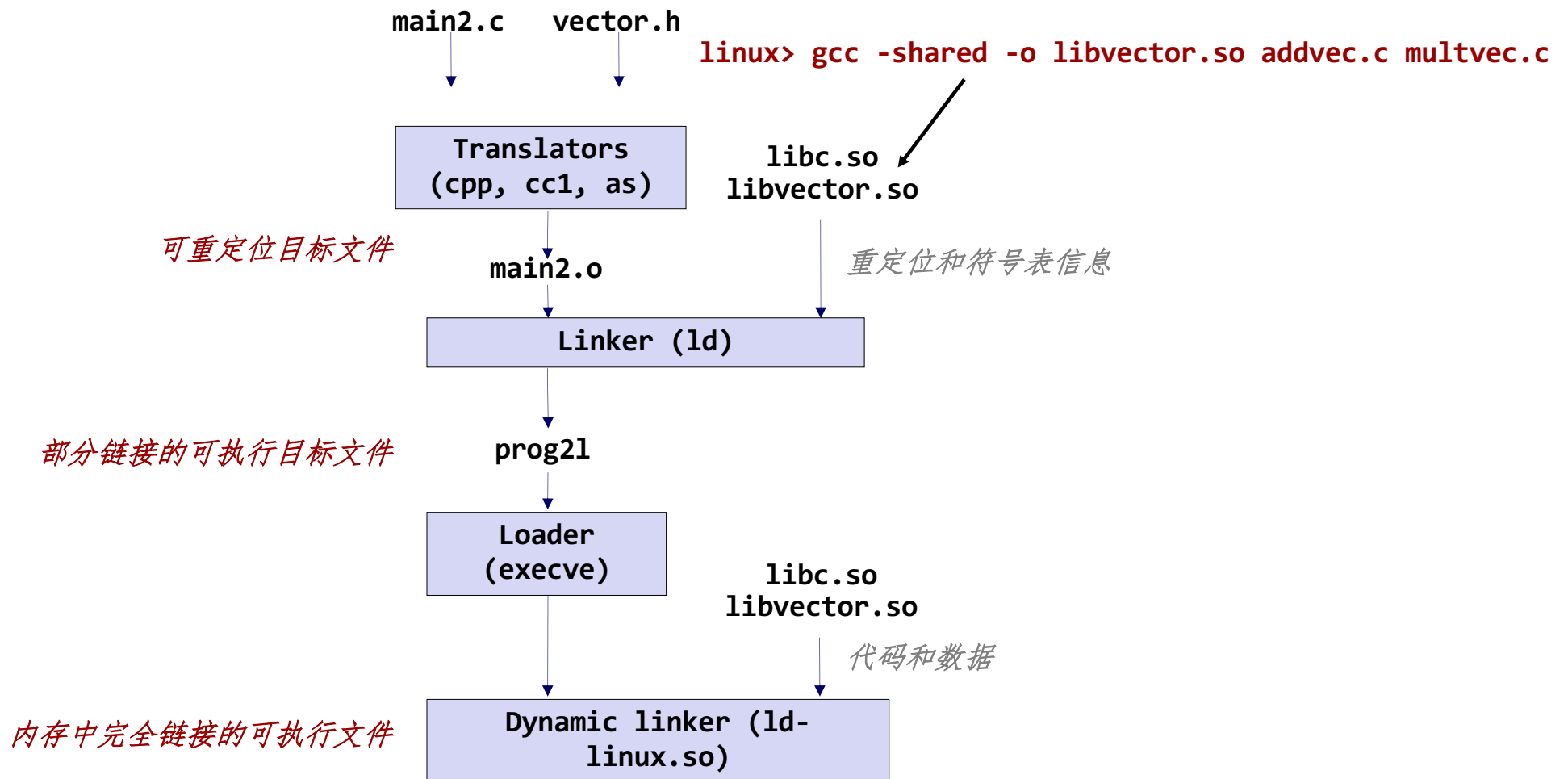
■ 现在的做法：共享库

- 包含代码和数据的目标文件，在加载时或运行时，动态地加载和链接到应用程序中
- 也被称为：动态链接库，DLL，`.so` 文件

共享库（续）

- 可以在可执行文件初次加载和运行时进行动态链接 (**load-time linking**)
 - Linux上最常见的情况，由动态链接器 (**ld-linux.so**) 自动处理
 - 标准 C 库 (**libc.so**) 通常是动态链接的
- 可以在程序开始执行后进行动态链接 (**run-time linking**)
 - 在Linux上，是通过调用 **dlopen()** 接口来实现的
 - 分布式软件
 - 高性能 web 服务器
 - 运行时库打桩
- 也可以多个进程共享库函数
 - 这个在学习了虚拟内存后，再深入理解

加载时动态链接



运行时动态链接

```
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

int x[2] = {1, 2};
int y[2] = {3, 4};
int z[2];

int main()
{
    void *handle;
    void (*addvec)(int *, int *, int *, int);
    char *error;

    /* Dynamically load the shared library that contains addvec() */
    handle = dlopen("./libvector.so", RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
}
```

dll.c

运行时动态链接

```
...

/* Get a pointer to the addvec() function we just loaded */
addvec = dlsym(handle, "addvec");
if ((error = dlerror()) != NULL) {
    fprintf(stderr, "%s\n", error);
    exit(1);
}

/* Now we can call addvec() just like any other function */
addvec(x, y, z, 2);
printf("z = [%d %d]\n", z[0], z[1]);

/* Unload the shared library */
if (dlclose(handle) < 0) {
    fprintf(stderr, "%s\n", dlerror());
    exit(1);
}
return 0;
}
```

dll.c

链接小结

- 链接是一项允许从多个目标文件构造程序的
- 链接可以发生在一个程序生命周期的各个阶段
 - 编译时（当编译一个程序时）
 - 加载时（当将程序加载进内存时）
 - 运行时（当执行程序时）
- 理解链接能够帮助你避免一些很讨厌的错误，使你成为一个更好的程序员